REINFORCEMENT LEARNING FOR BLOCK BLAST

From Deep Q-Networks to Deep Value Networks

March 2026

Nathan Duboisset

CONTENTS

1 Introduction **3**

2 Environment **3**

2.1 Game Rules . . . 3

2.2 Single-Piece Variant (1P) . . . 4

2.3 Three-Piece Variant (3P) . . . 4

2.4 Reward Shaping . . . 4

2.4.1 Initial Quadratic Reward (Abandoned) . . . 5

2.4.2 Final Reward Function (1P) . . . 5

2.4.3 Final Reward Function (3P) . . . 5

3 DQN-Based Approaches **5**

3.1 Architecture . . . 5

3.2 Double DQN (DDQN) . . . 6

3.3 Rainbow DQN . . . 6

3.4 Limitations . . . 6

4 Deep Value Network (DVN) **6**

4.1 Afterstate Formulation . . . 7

4.2 Architecture: Multi-Kernel CNN . . . 7

4.2.1 MLP Head . . . 7

4.3 Training . . . 8

4.4 3P Planner . . . 8

4.5 Results . . . 8

5 Discussion and Future Work **9**

6 Conclusion **10**

A Model Architectures **11**

A.1 BlockBlastCNNNet1P (DDQN / Rainbow) . . . 11

A.2 BlockBlastValueNet1PmultikernelFlattenned (DVN) . . . 11

Abstract

We investigate reinforcement learning approaches to the puzzle game *Block Blast*, played on an 8×8 grid. The agent must place Tetromino-like pieces to clear full rows and columns. We study two environment variants: a single-piece variant (1P) where one piece is given at a time, and a three-piece variant (3P) where three pieces are presented simultaneously and must all be placed in a freely chosen order. We experiment with classical Deep Q-Network approaches (DDQN, Rainbow DQN) and introduce a *Deep Value Network* (DVN) that operates on *afterstates*—the board states resulting from each possible placement—instead of learning a standard Q-function. The DVN achieves significantly better performance than naive baselines with only 42 461 parameters, compared to the ∼2.67M parameters needed by DQN-based models.

1 INTRODUCTION

Block Blast is a single-player puzzle game played on an 8×8 grid. At each turn, the player receives one or more Tetromino-like pieces and must place them on the grid. When a full row or column is formed, it is cleared and points are scored. The game ends when no more pieces can be placed.

This game presents several classic challenges for reinforcement learning:

- **Long-horizon planning**: clearing lines requires setting up the board over many turns.
- **Large action space**: up to 64 positions per piece in the 1P variant, 192 in the 3P variant.
- **Sparse rewards**: most individual placements yield no line clears.
- **Combinatorial structure**: in the 3P variant, the order of the three pieces matters.

We organised our experiments in three phases. First, we designed the environment and iterated on the reward function. Second, we trained standard DQN-based agents (DDQN and Rainbow DQN). Third, we developed a Deep Value Network (DVN) operating on afterstates.

2 ENVIRONMENT

2.1 Game Rules

The game is played on an 8×8 binary grid (0 = empty, 1 = filled). Seven piece shapes are used, each randomly rotated by a multiple of 90° at the start of each episode:

Shape	Description
Dot	1×1 single cell
Square	2×2 block
Big Square	3×3 block
Line-3	1×3 horizontal strip
Line-4	1×4 horizontal strip
L-small	2×2 L-shape
L-large	3×3 L-shape

After each placement, every row and column that is entirely filled is cleared simultaneously. The agent must avoid reaching a state where no valid placement exists, as this ends the episode.

2.2 Single-Piece Variant (1P)

In the 1P variant, one piece is drawn uniformly at random at the start of each step. The observation space is a dictionary containing:

- `board`: the 8×8 integer grid,
- `piece`: the piece shape encoded in a 5×5 padded grid,
- `valid_placements`: an 8×8 binary mask of valid top-left positions,
- `placements_result`: a precomputed $(8, 8, 8, 8)$ tensor giving the resulting board for every candidate placement, alongside an $(8, 8)$ reward map.

The action space is Discrete(64), where action $a = 8r + c$ places the piece with its top-left corner at row r, column c.

2.3 Three-Piece Variant (3P)

In the 3P variant, three pieces are drawn at the start of each *round*. The agent must place all three in a freely chosen order before a new round begins. The action space becomes Discrete(192):

$$a = \text{piece_idx} \times 64 + r \times 8 + c$$

A binary vector `pieces_used` tracks which pieces have been played. A `combo` counter accumulates cleared lines and is reset when a full round completes without any clearing.

The environment also exposes `get_t_plus_3_candidates(`γ`)`, which enumerates all 3! permutations and their valid positions, returning the terminal board, cumulative discounted reward, and action sequence for each candidate. This is used by the DVN 3P planner (Section 4).

2.4 Reward Shaping

2.4.1 • Initial Quadratic Reward (Abandoned)

In early experiments, we used a quadratic bonus for clearing multiple lines at once:

$$r = b \cdot n_{\text{cleared}}^2, \qquad b = 10$$

The intuition was to strongly incentivise combos. However, clearing multiple lines simultaneously is rare early in training, making the reward signal extremely sparse. The agent received near-zero reward for the vast majority of steps, causing training to stall.

2.4.2 • Final Reward Function (1P)

The final 1P reward is computed for each hypothetical afterstate:

$$r = \underbrace{r_{\text{survive}}}_{5.0} + \underbrace{b \cdot n_{\text{cleared}}}_{10\times\#\text{lines}} + \underbrace{b \cdot \rho}_{\text{emptiness}} - \underbrace{p_{\text{no3x3}}}_{20 \text{ if no free } 3\times3} \tag{1}$$

where $\rho \in [0, 1]$ is the fraction of empty cells (encourages keeping the board clean), and $p_{\text{no3x3}} = 20$ if no 3×3 window of empty cells exists after placement. Invalid moves incur a penalty of -100 and terminate the episode.

2.4.3 • Final Reward Function (3P)

The 3P variant uses a combo-based reward:

$$r = \begin{cases} 0.1 & \text{if no lines cleared} \\ b \cdot (n_{\text{cleared}}^2 + \text{combo}) & \text{if } n_{\text{cleared}} > 0 \end{cases}$$

The `combo` counter accumulates across placements, encouraging sustained clearing streaks.

3
DQN-BASED APPROACHES

3.1 Architecture

Both DDQN and Rainbow share the same convolutional backbone, `BlockBlastCNNNet1P`:

1. **Board branch**: two channels (board + valid mask), 8×8
   $\to$ Conv2d($2 \to 32$, $k = 3$) $\to$ Conv2d($32 \to 64$, $k = 3$) $\to$ Flatten $\to$ 4096 features.
2. **Piece branch**: one channel (5×5) $\to$ Conv2d($1 \to 32$, $k = 3$) $\to$ Flatten $\to$ 800 features.
3. **FC head**: $4896 \to 512 \to 256 \to 64$ Q-values.

Total parameters: **2 674 464**.

3.2 Double DQN (DDQN)

Double DQN decouples action selection from action evaluation in the target:

$$y_t = r_t + \gamma \cdot Q_{\theta^-}\Big(s_{t+1},\ \arg\max_a Q_\theta(s_{t+1}, a)\Big)$$

where θ is the policy network and θ^- the target network (updated every 10 episodes). Invalid actions are masked with -10^9 before the argmax.

Training: ε decayed $1.0 \to 0.05$ ($\times 0.9995$/episode), replay buffer 10 000, batch size 64, $\gamma = 0.99$, Adam lr $= 10^{-4}$.

3.3 Rainbow DQN

Rainbow combines five extensions:

- **Distributional RL (C51)**: outputs a probability distribution over 51 atoms in $[-10, 10]$; loss is KL divergence.
- **Prioritized Experience Replay (PER)**: priority $\propto |\delta|^\alpha$ ($\alpha = 0.5$), IS correction $\beta = 0.4$.
- **N-step returns**: 3-step bootstrapped targets.
- **Noisy Networks**: NoisyLinear layers ($\sigma_0 = 0.5$) replace ε-greedy.
- **Dueling architecture**: separate value and advantage streams, $Q = V + A - \bar{A}$.

3.4 Limitations

Despite 1 000 training episodes, neither DDQN nor Rainbow achieved consistent long-game performance:

1. **Credit assignment**: a good placement may not produce a line clear for several steps; discounted credit is weak.
2. **Q-function instability**: 64 outputs with a highly variable valid mask at each step.
3. **Sample efficiency**: 2.67M parameters trained on only 1 000 episodes is severely underfit.
4. **Perception overhead**: the network must simultaneously learn piece recognition, valid placements, *and* long-term board value.

4

DEEP VALUE NETWORK (DVN)

4.1 Afterstate Formulation

Instead of learning $Q(s, a)$, the DVN learns a *state value function* $V_\theta(s')$ where s' is the *afterstate*: the board after placement and line clearing. Since the environment precomputes `placements_result`, the agent enumerates all valid afterstates and selects:

$$a^* = \arg\max_a \left[r_a + \gamma \cdot V_\theta(s'_a)\right]$$

Advantages over Q-learning: the network only evaluates boards (not board–action pairs), valid constraints are handled implicitly, and the value function captures board quality independently of the current piece.

4.2 Architecture: Multi-Kernel CNN

The value network `BlockBlastValueNet1PmultikernelFlattenned` uses a bank of eight parallel convolutional branches:

Branch	Kernel	Filters	Wall padding	Output spatial
Square 1×1	$(1, 1)$	1	0	8×8
Square 2×2	$(2, 2)$	8	1	9×9
Square 3×3	$(3, 3)$	16	1	8×8
Square 4×4	$(4, 4)$	16	2	9×9
Square 5×5	$(5, 5)$	16	2	8×8
Square 8×8	$(8, 8)$	64	0	1×1
Horizontal 1×8	$(1, 8)$	4	0	8×1
Vertical 8×1	$(8, 1)$	4	0	1×8

Padding uses `ConstantPad2d` with value **1.0** so the kernel sees filled wall cells outside the board boundary. Line kernels (1×8, 8×1) use no padding.

Each branch: Flatten $\to$ Linear($\cdot \to 16$) $\to$ LeakyReLU $\to$ Linear($16 \to 8$). All eight outputs are concatenated into a **64-dimensional** feature vector.

The role of each branch family:

- 1×1: per-cell occupancy.
- 2×2–5×5: local block patterns matching common piece shapes.
- 8×8: global board summary.
- 1×8, 8×1: full-row and full-column detectors—crucial for identifying nearly-complete lines.

4.2.1 • MLP Head

$$\text{Linear}(64 \to 64) \to \text{LeakyReLU} \to \text{Linear}(64 \to 16) \to \text{LeakyReLU} \to \text{Linear}(16 \to 1)$$

Total parameters: **42 461** (63× fewer than DDQN/Rainbow).

4.3 Training

The DVN is trained via temporal difference learning on afterstates. Let s'_t be the chosen afterstate at time t.

TD Target. For non-terminal transitions:

$$y_t = \max_{a'} \left[r_{t+1,a'} + \gamma \cdot V_{\theta^-}(s'_{t+1,a'}) \right]$$

where θ^- is a target network updated every 200 episodes. For terminal transitions:

$$y_t = \frac{p_{\text{invalid}}}{\gamma}$$

Loss. Huber loss (SmoothL1) with gradient clipping at norm 10.

Protocol.

- 10 000 episodes, max 100 steps/episode.
- ε-greedy: $1.0 \to 0.01$, decay $\times 0.999$/episode.
- Replay buffer: 100 000 transitions, batch size 128.
- Adam, lr $= 10^{-4}$, StepLR ($\gamma_{\text{sched}} = 0.99$, step 1 000).
- Checkpoints every 500 episodes with full training state; logging via Weights & Biases.

4.4 3P Planner

For the 3P variant, a `RoundPlanner3P` wraps the DVN. At each new round it calls `get_t_plus_3_candidates(`γ`)`, scores each candidate by computing the immediate discounted rewards plus the discounted value:

$$\text{score} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 \cdot V_\theta(s_{t+3})$$

and executes the best 3-action sequence, replanning if an action becomes invalid.

4.5 Results

Figure 1 shows the episode return and length distributions for the trained DVN, a greedy baseline, and a random baseline.

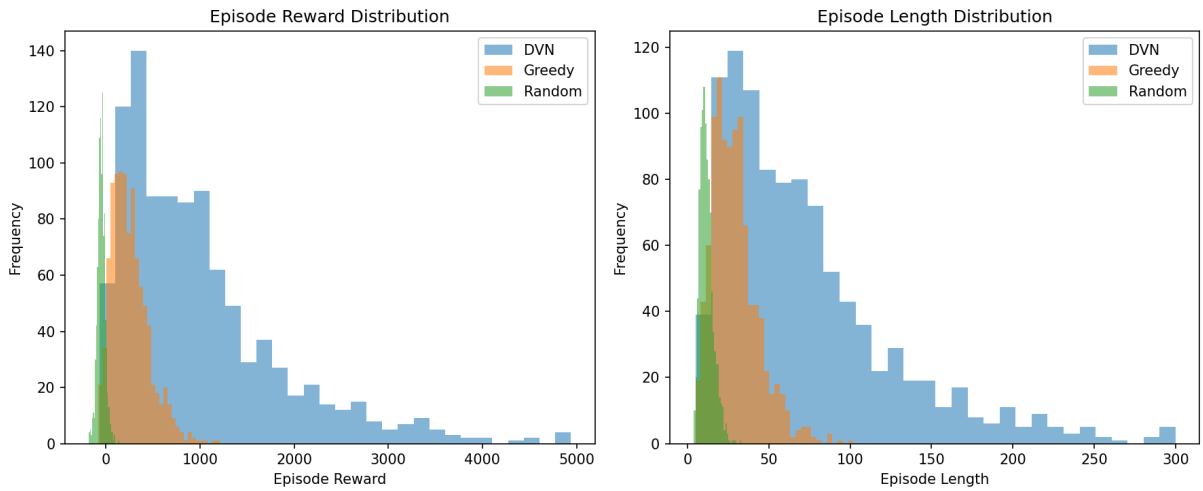

Figure 1: Distribution of episode returns (left) and episode lengths (right) for DVN, greedy, and random agents on the 1P Block Blast environment.

The trained DVN manages to play very good games (reaching around 300 steps in the best cases), giving it a high mean score. However, it exhibits a high variance with a median that is significantly lower than the mean. This is primarily due to the highly stochastic nature of the environment, which inevitably leads to a large number of poorly performing "bad" games.

Agent	Avg. Return	Avg. Steps	Parameters
Random	—	∼5	—
Greedy	—	∼30	—
DVN (untrained)	−815.98	∼20	42 461
DVN (trained, 60 avg)	—	∼60	42 461
DDQN 1P	—	∼20	2 674 464
Rainbow 1P	—	∼20	2 674 464

Table 1: Agent performance summary on the 1P environment.

The trained DVN achieves ∼60 average steps per episode vs. ∼20 for an untrained model. The greedy baseline (always pick highest immediate reward) is shortsighted and over-fills the board; the DVN outperforms it by learning to preserve board space.

5 DISCUSSION AND FUTURE WORK

Why DVN outperforms DQN. The DVN decouples *what the board looks like after the move* from *which action achieves it*. The value network only needs to learn a board-to-value mapping—far simpler than the full Q-function which also encodes piece geometry and placement logic.

Limitations.

- The value function sees only the afterstate board, *not* upcoming pieces.
- The 3P planner's full enumeration is $O(3! \times 64^3)$ in the worst case—slow without batched GPU acceleration.
- The environment's piece generation is purely random and thus highly stochastic, unlike the real game which employs logic to avoid giving overly impossible sequences (e.g., three 3×3 blocks consecutively). This means our environment is unnecessarily harsh on the agent, whose actual performance on the real game might be significantly better.

Future directions.

- Include next-piece information as extra input channels.
- Add a hand-crafted feature branch (holes, row/column fill density, height variance).
- Batch the 3P lookahead on GPU.
- Monte Carlo Tree Search guided by the value function.

6 CONCLUSION

We have presented a reinforcement learning study of Block Blast. Standard DQN approaches (DDQN, Rainbow, ∼2.67M parameters) struggled to converge within the training budget. A Deep Value Network operating on afterstates—with only 42 461 parameters and a multi-kernel CNN architecture—achieved substantially better performance. The main advantage of the DVN approach is that it does not need to learn the basic mechanics and geometry of the game, since these are handled implicitly by the afterstate generation. However, its true limitation remains the high variance across episodes. Ultimately, what truly allowed the agent to progress from an average of ∼30 steps to ∼60 steps was the proper design and tuning of the reward shaping.

A
MODEL ARCHITECTURES

A.1 BlockBlastCNNNet1P (DDQN / Rainbow)

```
Board + Valid Mask (2, 8, 8)
  -> Conv2d(2->32, k=3, pad=1) -> ReLU
  -> Conv2d(32->64, k=3, pad=1) -> ReLU
  -> Flatten -> 4096

Piece (1, 5, 5)
  -> Conv2d(1->32, k=3, pad=1) -> ReLU
  -> Flatten -> 800

Concat [4096 + 800] = 4896
  -> Linear(4896->512) -> ReLU
  -> Linear(512->256) -> ReLU
  -> Linear(256->64)   [Q-values for 64 actions]
```

A.2 BlockBlastValueNet1PmultikernelFlattenned (DVN)

```
Board (1, 8, 8) -- 8 parallel branches:

  Branch k: ConstantPad2d(pad_k, val=1.0)
         -> Conv2d(1->filters_k, kernel_k)
         -> LeakyReLU -> Flatten
         -> Linear(flat_k -> 16) -> LeakyReLU
         -> Linear(16 -> 8)

Concat [8 branches x 8] = 64
  -> Linear(64->64) -> LeakyReLU
  -> Linear(64->16) -> LeakyReLU
  -> Linear(16->1)   [V(s')]
```